

Adding a New Training Model

This guide explains how to add a new model to the connectome-gnn training pipeline. The codebase uses a registry pattern: you write a model class, register it under a name, and create a YAML config referencing that name. The trainer picks it up automatically.

Architecture Overview

The pipeline has three layers. The **config** (`config/{biomodel}/{name}.yaml`) defines the dataset and model name. The **registry** (`src/connectome_gnn/models/registry.py:42`) maps `signal_model_name` strings to model classes. The **trainer** (`src/connectome_gnn/models/graph_trainer.py:84-91`) dispatches to the right training loop based on the model name.

Two training loops exist. `data_train_gnn` (line 96) handles GNN and Linear ODE models that take (`NeuronState`, `edge_index`) as input. `data_train_gnn_RNN` (line 853) handles sequential models (RNN, LSTM) that take a packed state tensor and maintain hidden state across time steps. The dispatcher at line 86 checks if 'rnn' or 'lstm' appears in the signal model name to decide which loop to use.

Step 1: Write the Model Class

Create a new file in `src/connectome_gnn/models/`, for example `my_model.py`. Your model must be an `nn.Module` with a `forward` method. The forward signature depends on which training loop you target.

For the GNN-style loop (`data_train_gnn`), the forward method receives a `NeuronState` object and an `edge_index` tensor. `NeuronState` (defined in `src/connectome_gnn/neuron_state.py`) holds per-neuron fields: `voltage`, `stimulus`, `index`, `neuron_type`, etc. Your model should return `dv/dt` as a `(N, 1)` tensor. See `flyvis_linear.py` for a minimal example (124 lines) or `neural_gnn.py` for the full GNN.

For the RNN-style loop (`data_train_gnn_RNN`), the forward method receives a packed state tensor `x` of shape `(N, features)` where voltage is at column 3 and stimulus at column 4. It also receives a hidden state `h` and returns `(dv_dt, h_next)` when `return_all=True`. See `neural_rnn.py` for the GRU-based implementation.

Both model types must expose two attributes for connectivity benchmarking: `self.W` (an `nn.Parameter` of shape `(n_edges, 1)` representing per-edge synaptic weights) and `self.a` (an `nn.Parameter` of shape `(n_neurons, embedding_dim)` representing per-neuron embeddings). These are compared against ground truth in `GNN_PlotFigure.py` to compute `connectivity_R2` and `cluster_accuracy`. If your model doesn't naturally learn per-edge weights, initialize `self.W` as a learnable parameter anyway (see `neural_rnn.py:71-83` for how the RNN does this).

The constructor receives (`aggr_type`, `config`, `device`) where `config` is the full config object. Extract what you need from `config.simulation`, `config.graph_model`, and `config.training`. Key fields: `n_neurons`, `n_edges`, `n_extra_null_edges` (from `simulation`), `hidden_dim`, `n_layers`, `embedding_dim` (from `graph_model`), `w_init_mode`, `batch_size` (from `training`).

Step 2: Register the Model

Add the `@register_model` decorator to your class with one or more name strings. These names will match the `signal_model_name` field in YAML configs.

```
from connectome_gnn.models.registry import register_model
@register_model("my_model", "drosophila_cx_mymodel",
               "larva_mymodel")
class MyModel(nn.Module):
```

```
...
```

Then add the import to the discovery function in `src/connectome_gnn/models/registry.py:18-26`:

```
def _discover_models():
    global _discovered
    if _discovered:
        return
    _discovered = True
    import connectome_gnn.models.neural_gnn
    import connectome_gnn.models.flyvis_linear
    import connectome_gnn.models.neural_rnn
    import connectome_gnn.models.my_model # <-- add this line
```

Step 3: Create a Config YAML

Create a config file in `config/{biomodel}/`. The critical field is `graph_model.signal_model_name` which must match one of your registered names. Here is a minimal example for `drosophila_cx`:

```
description: "My model baseline on drosophila_cx"
dataset: drosophila_cx_mymodel
simulation:
    # ... copy from drosophila_cx.yaml (keep identical)
graph_model:
    signal_model_name: drosophila_cx_mymodel # <-- matches
@register_model
prediction: first_derivative
hidden_dim: 64
n_layers: 3
embedding_dim: 2
# ... other architecture params
training:
    n_epochs: 20
    batch_size: 2
    data_augmentation_loop: 500
    lr_W: 0.001
    lr: 0.001
    w_init_mode: zeros
# ... other training params
```

The `dataset` field determines where data is loaded from and where logs are saved. If your model uses the same data as the GNN (same connectome, same stimulus), point `dataset` to an existing dataset name like `drosophila_cx` and the trainer will load the pre-generated data. If you use a new dataset name (e.g. `drosophila_cx_mymodel`), data generation will create a new folder under `graphs_data/`.

The routing function `add_pre_folder` in `src/connectome_gnn/utils.py:314` maps dataset names to config directories using prefix matching. Any dataset starting with `drosophila_cx` routes to `config/drosophila_cx/`. Same for `larva` and `zebrafish_oculomotor`.

Step 4: Training

If your model is a GNN-style model (not RNN), the trainer at `graph_trainer.py:527` calls:

```
pred, in_features, msg = model(batched_state, batched_edges,
```

```
data_id=data_id, return_all=True)
```

If it's an RNN-style model, the trainer at `graph_trainer.py:902` calls:

```
pred, h = model(x, h=h, return_all=True)
```

To route your model to the RNN loop, include 'rnn' or 'lstm' in the `signal_model_name`. Otherwise it goes through the GNN loop.

Run training with:

```
python GNN_train.py -o generate_train_test_plot
drosophila_cx_mymodel
```

Or through the LLM exploration pipeline:

```
python GNN_LLM.py -o generate_train_test_plot_Claude
drosophila_cx_mymodel iterations=128 --cluster
```

Step 5: Analysis and Plotting

`GNN_PlotFigure.py` handles all post-training analysis. It extracts `model.W` to compute `connectivity_R2` against ground truth, `model.a` for clustering, and runs rollout prediction. The analysis code checks `signal_model_name` for routing at several points (lines 661, 666, 740) · if your model name contains 'linear', 'MLP', or 'rnn', it uses model-specific plotting paths. Otherwise it falls through to the GNN plotting path.

Existing Models

| `signal_model_name` | Class | File | Training loop | Description | |---|---|---|---| | `drosophila_cx`, `larva`, `zebrafish_oculomotor`, `flyvis_A` | `NeuralGNN` | `neural_gnn.py` | GNN | Message-passing GNN with learnable `g_phi` and `f_theta` MLPs | | `flyvis_linear`, `drosophila_cx_linear`, etc. | `LinearODE` | `flyvis_linear.py` | GNN | Linear ODE: $dv/dt = (-v + W \cdot \text{relu}(v) + I + V_{\text{rest}}) / \tau$ | | `flyvis_rnn`, `drosophila_cx_rnn`, etc. | `NeuralRNN` | `neural_rnn.py` | RNN | GRU-based RNN, flat input (no graph structure) |

The GNN model also supports Neural ODE training via the flag `training.neural_ode_training: true` in the config. This wraps the GNN forward pass with `torchdiffeq.odeint_adjoint` for O(1) memory rollout training. The wrapper lives in `neural_ode_wrapper.py` and is invoked at `graph_trainer.py:536-564`.

Key Files

| File | Purpose | |---|---| | `src/connectome_gnn/models/registry.py` | Model registry: `@register_model` decorator and `create_model()` lookup | | `src/connectome_gnn/models/graph_trainer.py` | Training loops: `data_train_gnn` (line 96) and `data_train_gnn_RNN` (line 853) | | `src/connectome_gnn/models/neural_ode_wrapper.py` | Neural ODE adjoint wrapper for any model | | `src/connectome_gnn/neuron_state.py` | `NeuronState` dataclass (voltage, stimulus, index, etc.) | | `src/connectome_gnn/config.py` | Config dataclasses (`SimulationConfig`, `GraphModelConfig`, `TrainingConfig`) | | `src/connectome_gnn/utils.py:314` | `add_pre_folder`: dataset name to config directory routing | | `GNN_PlotFigure.py` | Post-training analysis: `connectivity_R2`, rollout, clustering | | `GNN_train.py` | Entry point for standalone training | | `GNN_LLM.py` | Entry point for LLM-guided exploration |